ε-Sufficient Context: Definitions for Code Editing by Bounded Readers

Jude Castiel Johansen Independent researcher castiel@shockware.co.uk ORCID 0009-0002-5644-

October 2026

Abstract

Retrieving context for a language model that edits code is usually evaluated by end-task pass rate. There is no agreed definition of what context is *sufficient*, and the classical notion from program slicing assumes a lossless reader for whom more context never hurts. That assumption fails for language models, whose accuracy falls as input grows. This note fixes definitions. A change request is a contract (Δ, D, I) with a frame condition. A context K is *ε-sufficient* for an editor A if A's success on K is within ε of its success on the best context, not on the full repository. Under a coverage-times-degradation model of the editor we prove, and mechanise in Lean 4, that ε-sufficiency is not closed under superset, while it is closed under superset when the reader is lossless. We state the forward slice as a frame requirement, define *context-closed* languages by three properties, and describe a planned benchmark, SuffBench, whose instances carry ground-truth minimal contexts. No experiments are reported; the purpose is to timestamp the definitions.

1 Problem

A code change is produced by an editor from a request and some portion of the repository. When the editor is a compiler or an unhurried human, the portion can be large without penalty, and the right question is only whether it contains enough. Program slicing [11, 4] answers that question: the slice is the minimal set of statements that can affect a criterion, and any superset of a sufficient set is sufficient.

When the editor is a language model, the portion is read under a budget and with attention that degrades as the input grows [5, 9]. More context can lower success. The slicing notion of minimality therefore targets the wrong object. The gap this note addresses is the following: no existing definition of minimal context is stated relative to a reader whose accuracy is non-monotone in input size.

2 Setting

A codebase is a graph $G = (U, E)$. U is a finite set of *units*, such as files, functions, or classes. E is a set of typed edges, such as calls, imports, data flow, inheritance, and tests-cover. A program P assigns a body to each unit. Its observable behaviour is a map $[\![P]\!] : X \to Y$ from inputs to outputs or traces. Φ is a set of predicates over programs, including tests, type judgements, and stated invariants. An *oracle* $V \subseteq \Phi$ is the set of checks actually run on a candidate program; $V(P') = \mathsf{pass}$ means every check in V holds of P'.

3 The contract

Definition 1 (Contract)**.** A change request is a triple $C = (\Delta, D, I)$ with $\Delta \subseteq \Phi$ the requested behaviour, $D \subseteq X$ the scope of inputs on which behaviour may differ, and $I \subseteq \Phi$ the invariants. A program P' *satisfies* C relative to the current program P, written $P' \models C$, iff

(a)	$\forall \varphi \in \Delta.\ \varphi(P')$	requested behaviour
(b)	$\forall x \notin D.\ [\![P']\!](x) = [\![P]\!](x)$	frame: nothing else changes
(c)	$\forall \varphi \in I.\ \varphi(P) \Rightarrow \varphi(P')$	invariants preserved.

C is *well-defined* iff some P' satisfies it, and *precise* iff any two satisfying programs agree on D. Imprecision is the formal name for an ambiguous request. Condition (b) is the frame condition in the sense of Borgida, Mylopoulos and Reiter [3] and of the frame rule in separation logic [6]. It is the condition most specifications omit and the one that context reduction endangers.

4 Sufficiency for a bounded, stochastic editor

A *context* is a subset $K \subseteq U$. An *editor* A maps a contract and a context to a probability distribution over edits; an edit rewrites a write-set $W \subseteq K$ and yields a program $A(C, K)(P)$. We write the editor's success rate as

$$p_A(K, C) \;=\; \Pr\big[\,V\big(A(C, K)(P)\big) = \mathsf{pass}\,\big], \qquad p_A^*(C) \;=\; \max_{K \subseteq U} p_A(K, C).$$

Definition 2 (ε-sufficient context)**.** K is (ε, A)-*sufficient* for C iff $p_A(K, C) \geq p_A^*(C) - \varepsilon$. Given a cost function on contexts, K is *minimal* iff it is sufficient and no K' with $\mathrm{cost}(K') < \mathrm{cost}(K)$ is sufficient.

Three points distinguish this from the slicing notion. The editor is a distribution, not a function, so sufficiency is a tolerance. The comparison is to the best context, not to the full repository, because the full repository may itself be a poor choice. And nothing in the definition makes sufficiency closed under superset.

Assumption 1 (Coverage and degradation)**.** There is an *oracle slice* $O(C) \subseteq U$, the units whose contents determine the verdict of V on the change, and the editor's success factors as

$$p_A(K, C) \;=\; r(K, C) \cdot g_A(\mathrm{load}(K)),$$

where $r(K, C) \in [0, 1]$ is coverage of the oracle slice, in the simplest form $r = 1$ if $O(C) \subseteq K$ and 0 otherwise, and g_A is the reader's *degradation curve*, a non-increasing function of context load.

Coverage is a property of the repository and the contract. The degradation curve is a property of the model, estimable once per model with planted-fact experiments independent of any codebase. Under the assumption, the optimum is the smallest superset of the oracle slice, and sufficiency reduces to two checks: coverage holds, and degradation at that load is within ε.

The tolerance ε cannot be removed. The editor is stochastic, so no context gives certainty. The degradation curve is estimated from n samples, contributing a statistical term of order $\sqrt{\ln(1/\delta)/(2n)}$ by Hoeffding's inequality. And the oracle V is incomplete: tests passing is not correctness. The third term is unmeasurable and should always be reported as such.

5 Non-monotonicity

We make Assumption 1 concrete in its simplest form and show that sufficiency is not closed under superset, in contrast with the lossless case. Both statements are mechanised in Lean 4 with Mathlib.[1]

Fix a finite type of units, an oracle slice O, and a degradation curve $g : \mathbb{N} \to \mathbb{Q}$ with $g \geq 0$. Define $p(K) = g(|K|)$ if $O \subseteq K$ and $p(K) = 0$ otherwise. If g is non-increasing then $p(K) \leq g(|O|)$ for every K, with equality at $K = O$, so $p^* = g(|O|)$. Say K is ε-sufficient iff $p(K) \geq g(|O|) - \varepsilon$.

Proposition 1 (Sufficiency is not closed under superset)**.** *Let g be strictly decreasing. For every unit $u \notin O$ and every ε with $0 \leq \varepsilon < g(|O|) - g(|O| + 1)$, the context O is ε-sufficient and the context $O \cup \{u\}$ is not.*

Proof. $O \subseteq O$, so $p(O) = g(|O|) \geq g(|O|) - \varepsilon$ because $\varepsilon \geq 0$. For $K' = O \cup \{u\}$ with $u \notin O$ we have $O \subseteq K'$ and $|K'| = |O| + 1$, so $p(K') = g(|O| + 1) < g(|O|) - \varepsilon$ by the bound on ε. Hence K' is not ε-sufficient. □

Lemma 1 (The lossless case is closed under superset)**.** *Let g be constant with value $c \geq 0$. If K is ε-sufficient and $K \subseteq K'$, then K' is ε-sufficient.*

Proof. If $\varepsilon \geq c$ every context is sufficient. Otherwise $p(K) \geq c - \varepsilon > 0$ forces $O \subseteq K \subseteq K'$, so $p(K') = c \geq c - \varepsilon$. □

The pair is the point. The classical reader makes sufficiency monotone and minimality a matter of removing what is unnecessary. The bounded reader makes adding the unnecessary a failure mode, so minimality becomes an optimisation with a cost that is non-monotone in benefit.

[1] File `sufficiency/Sufficiency/NonMonotonic.lean` at commit of `https://github.com/Shockware-Limited/context-sufficiency-note`. The development has no `sorry`.

6 The forward slice as a frame requirement

Let W be the write-set of an edit. The *backward slice* of W is the set of units W depends on, through calls, imports, and data flow. The *forward slice* of W is the set of units that depend on W. Retrieval for code editors has concentrated on the backward slice: what the edit site reads and calls [10, 13, 12, 1].

Condition (b) of the contract can be violated only through the forward slice. A unit that neither depends on W nor is in W has the same body and the same dependencies before and after the edit, so its behaviour is unchanged. Every frame violation therefore lives in a dependent of W, and the tests that would catch it are the tests bound to those dependents, as in safe regression test selection [8] and change impact analysis [7].

Requirement 1 (Forward slice). A context selector that omits the dependents of W cannot witness a violation of condition (b). Any claim that an edit produced from such a context satisfies the contract rests entirely on the oracle, not on the context.

This is stated as a requirement and not a theorem because it depends on the dependency relation being exact. In languages with reflection, dynamic dispatch, or duck typing, the static forward slice under-approximates the true one, and the requirement can be met in appearance but not in fact.

7 Context-closed languages

Semantic sufficiency is undecidable in general. It becomes decidable when the language restricts programs so that the quantities above are graph queries. We state three properties as conjectures to be proved for a core calculus; the calculus itself is future work and no language is named here. Write $\mathrm{spec}(v)$ for the declared interface of unit v, including its contract and effects, and $\mathrm{deps}(u)$ for the units u depends on.

Conjecture 1 (P1, interface determinacy)**.** $[\![u]\!] = F\big(\mathrm{body}(u), \{\mathrm{spec}(v) : v \in \mathrm{deps}(u)\}\big)$ *for some fixed* F*. The meaning of a unit depends on its dependencies only through their interfaces.*

Conjecture 2 (P2, spec-bounded reliance)**.** *If an edit to* u *leaves* $\mathrm{spec}(u)$ *unchanged and the new body satisfies* $\mathrm{spec}(u)$*, then the verdict of* V *on every other unit is unchanged.*

Conjecture 3 (P3, computable sufficiency)**.** *For a contract* C *with write-set* W*, the context*

$K^* = W \cup \{\mathrm{spec}(v) : v \in \mathrm{deps}(W)\} \cup \{u \in \mathrm{dependents}(W) : \mathrm{spec}(W)\ \textit{changes}\} \cup$ *tests bound to the above*

satisfies the coverage condition of Assumption 1 by construction, and is computable in time linear in $|G|$*.*

Definition 3 (Context-closed)**.** A language is *context-closed* iff P1, P2 and P3 hold of all its programs.

In a context-closed language the backward slice stops at one hop of interfaces, the forward slice is provably empty when the interface is preserved, and the undecidable part of sufficiency disappears. The tolerance ε then measures only the editor's degradation, never the graph. Individually, the ingredients exist: declared contracts, declared effects, content-addressed definitions, acyclic modules, and language-independent slicing by observation [2]. The property is what is new.

8 Proposed benchmark

We reserve the name *SuffBench* for a benchmark whose instances are tuples (repository snapshot, contract C derived from a fail-to-pass commit, gold write-set, oracle slice O). The oracle slice is to be computed by observation: stub each unit outside the gold write-set, rebuild, rerun the checks in V, and retain the unit iff the verdict changes, with delta debugging to reduce the number of builds. Instances must postdate the training cutoff of any evaluated model.

The dataset labels *verification* sufficiency: what determines the verdict. Experiments measure *generation* success: whether an editor given K produces a passing edit. The two must not be conflated. Any claim of the form "less context without loss" is to be stated as "within ε of the best observed pass rate under oracle V" with the oracle residual reported as unmeasured. Nothing is released with this note.

9 Related work

Slicing [11, 4] and its language-independent, observation-based form [2] define minimal context for a lossless consumer. The frame condition appears as a specification problem in [3] and as a proof rule in [6]. Safe regression test selection [8] and change impact analysis [7] compute which checks a change can affect, which is the forward-slice half of the contract. For language-model editors, context is selected by learned prompt proposals [10], iterative retrieval [13], simple localisation [12], and dependency-graph planning [1]; none states a sufficiency criterion, and all compare against task pass rate. That more context can hurt a language model is established empirically [5, 9] but has not been carried into a definition of minimal context.[2] The gap, restated: no prior definition of minimal context is relative to a reader whose accuracy is non-monotone in input size.

References

[1] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.

[2] David Binkley, Nicolas Gold, Mark Harman, Syed Islam, Jens Krinke, and Shin Yoo. ORBS: Language-independent program slicing. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*, pages 109–120, 2014.

[3] Alexander Borgida, John Mylopoulos, and Raymond Reiter. On the frame problem in procedure specifications. *IEEE Transactions on Software Engineering*, 21(10):785–798, 1995.

[4] Susan Horwitz, Thomas Reps, and David Binkley. Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems*, 12(1):26–60, 1990.

[5] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[6] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL 2001)*, volume 2142 of *Lecture Notes in Computer Science*, pages 1–19, 2001.

[7] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2004)*, pages 432–448, 2004.

[8] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997.

[9] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227, 2023.

[10] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. Repository-level prompt generation for large language models of code. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31693–31715, 2023.

[11] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. Describes the Agentless approach.

[2] A small open-source project named MinCtx, by Zane Yuan on GitHub, carries the slogan "minimum context, equivalent outcome"; it is adjacent in aim but offers no formal treatment.

[13] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pages 2471–2484, 2023.